



LOST BOYS CYBER

THREAT INTELLIGENCE • MALWARE ANALYSIS • DETECTION ENGINEERING



CVE-2026-7774 tarfile.data_filter path traversal bypass allows writing outside the extraction directory

CVE / Vulnerability Threat Intelligence Report

Classification	TLP:CLEAR
Published	2026-06-15
Version	1.1
Author	Lost Boys Cyber
Report Type	CVE / Vulnerability Threat Intelligence Report

TLP:CLEAR | Lost Boys Cyber | CVE-2026-7774 tarfile.data_filter path traversal bypass allows writing outside the extraction directory - CVE / Vulnerability Threat Intelligence Report

Published: 2026-06-15 | TLP:CLEAR | Version: 1.1 | Author: Lost Boys Cyber

AI Generation: This report was generated with AI assistance and is provided for informational purposes only. All findings, IOCs, detection rules, hunting queries, attribution assessments, and recommendations must be independently reviewed and validated by a qualified security professional before operational use.

Table of Contents

1. Executive Summary
2. Intelligence Requirement
3. Source Review and Confidence
4. Key Findings
5. Threat Context
6. Detection and Hunting
7. Recommendations
8. References

CVE-2026-7774 tarfile.data_filter path traversal bypass allows writing outside the extraction directory

1. Executive Summary

CVE-2026-7774 is a medium-severity path-traversal flaw in CPython's `tarfile.data_filter`` safety control. Public CNA, NVD, Python security-announce, upstream issue/patch, and downstream Debian sources agree that crafted link entries can bypass the filter and redirect later archive members outside the intended extraction directory. In practical terms, a malicious tar archive can make `tarfile.extractall()` write files outside the destination path with the permissions of the extracting process.

This is not a classic remotely reachable network-service bug. Exploitation requires a victim-controlled extraction event, such as a developer unpacking a third-party tarball, a CI runner processing source artifacts, or an application using Python archive extraction on attacker-supplied content. The integrity impact is still meaningful because the flaw turns a supposedly safer extraction mode into an arbitrary out-of-directory write primitive.

The strongest immediate defensive actions are to patch to fixed Python releases, reduce trust in unvetted tar archives, and instrument archive-extraction workflows that touch untrusted content. Defenders should also note the Python documentation's explicit warning that even `filter='data'` is not sufficient for safely extracting untrusted archives without additional controls.

2. Intelligence Requirement

Intelligence Requirement	Assessment
What is the vulnerability?	A bypass of <code>tarfile.data_filter`</code> in CPython that allows crafted link entries, including empty-name or directory-like symlinks, to redirect later extracted files outside the intended destination directory.
What is the business risk?	Integrity risk to developer workstations, CI/CD runners, internal package-processing services, and any product feature that extracts attacker-controlled tar archives with Python.
Who is exposed?	Organizations running affected CPython releases or downstream packages that embed the vulnerable <code>tarfile`</code> logic in archive-handling workflows.
What should defenders do next?	Upgrade to fixed Python versions or distro packages, constrain archive extraction, avoid unnecessary privileges, and add telemetry for Python-driven archive extraction followed by unexpected writes.

3. Source Review and Confidence

Source	Contribution	Confidence
Microsoft Security Response Center advisory	Publication tracking and intake reference for CVE-2026-7774.	Medium
CVE Project JSON 5 / CNA record	Canonical title, affected-version boundaries, credits, and reference set from the Python Software Foundation.	High
NVD JSON / NVD detail page	CVSS v4.0 score, CWE mapping, and public status.	High
Python <code>security-announce`</code> thread	Vendor-adjacent disclosure text summarizing impact and attack	High

	preconditions.	
CPython issue `#149486`	Describes the empty-named symlink chain bypass condition.	High
CPython PR `#149487` and backport commits	Highest-confidence technical remediation source for the actual fix logic and regression tests.	High
Python 3.14.6 / 3.13.14 / 3.15.0b2 changelog entries	Confirms the fix shipped in maintained upstream release lines.	High
Python `tarfile` documentation	Confirms that `filter='data'` still is not a complete untrusted-archive defense even after the fix.	High
Debian Security Tracker	Corroborates downstream impact and shows distro-package status across supported Debian releases.	High

Confidence is high on the core mechanics, severity band, and affected/fixed release boundaries because the reviewed material is anchored in the PSF CNA record and the upstream CPython patch set. Confidence is lower only on exploitation prevalence: the reviewed public sources did not provide evidence of broad in-the-wild exploitation or CISA KEV inclusion at the time of review.

4.1 Vulnerability Metadata

Field	Value
CVE	`CVE-2026-7774`
Product / Component	CPython `tarfile` module, specifically `tarfile.data_filter`-mediated extraction behavior
Weakness	`CWE-22 Path Traversal`
Severity	`CVSS v4.0 6.9 / MEDIUM`
CVSS vector	`CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:A/VC:N/VI:H/VA:N/SC:N/SI:N/SA:N`
Vulnerable behavior	Crafted link entries can redirect later archive members outside the intended extraction directory even when `data_filter` is used
Affected versions (CNA record)	`0` before `3.13.14`, `3.14.0` before `3.14.6`, and `3.15.0a1` before `3.15.0b2`
Fixed upstream versions	`3.13.14`, `3.14.6`, and `3.15.0b2`
Public disclosure	2026-06-04
Reporter / credits	Phùng Siêu Đạt (OPSWAT Unit 515); remediation and coordination credits include Gregory P. Smith, Petr Viktorin, Seth Larson, and Stan Ulbrych

4.2 Technical Findings

Finding	Detail	Why It Matters
---------	--------	----------------

Link-target validation mismatch	Upstream commit <code>`578411982c16f753f4893532510099ef665117da`</code> states that <code>`data_filter`</code> normalized link names for writing, but performed the containment check against the unnormalized value.	A crafted archive could pass validation but still write a normalized path outside the destination.
Trailing-slash symlink handling flaw	The patch also notes that the vulnerable code computed a symlink's directory before stripping trailing separators from the member name.	Directory-like names and crafted link entries could be resolved relative to the wrong location.
Destination-directory replacement edge case	The fix explicitly rejects link members that resolve to the destination directory itself.	Without that check, an attacker could replace the destination with a symlink and redirect all later members.
Empty-name / directory-like symlink chains	The linked CPython issue documents bypass using empty-named symlink chains and unusual link-member layouts.	This widens exploitation beyond simple <code>`../`</code> abuse and raises the chance that higher-level archive-processing code misjudges its exposure.

The upstream remediation pattern is important for defenders reviewing custom archive logic. The fix normalizes the path actually written to disk, strips trailing separators before calculating the containing directory for symlinks, resolves the target path using ``realpath(..., strict=os.path.ALLOW_MISSING)``, and blocks any target that escapes the extraction root or resolves to the root itself.

4.3 Exposure and Impact Assessment

Exposure Pattern	Likely Impact	Relative Risk
Developer manually extracts third-party or customer-provided tar archive	Arbitrary overwrite or file placement outside the intended working directory within the user's write permissions	High
CI/CD runner unpacks source bundles, build artifacts, or dependency tarballs	Integrity impact to build outputs, workspace files, or auxiliary scripts reachable by the build user	High
SaaS / internal service extracts customer-supplied tar content with Python	Multi-tenant or service-side file-write impact depending on sandboxing and filesystem permissions	High
Sandboxed extraction to disposable temp directory with strict permissions	Blast radius reduced to the sandbox scope; persistence opportunities are constrained	Medium-Low
Fully patched interpreter or distro package with additional archive-validation controls	Vulnerable primitive removed and residual risk limited to generic tar DoS / content risks	Low

4.4 ATT&CK-Relevant Defensive Framing

ATT&CK Technique	Name	Rationale	Confidence
T1204.002	User Execution: Malicious File	Exploitation requires a user, service, or automation workflow to process an attacker-controlled tar archive.	Medium

No stronger ATT&CK mapping is assigned in this review because the public evidence describes a vulnerability and likely delivery path, not a fully documented intrusion chain or recurring actor tradecraft pattern.

4.5 Indicator Assessment

Indicator Category	Assessment
Stable network IOC set	None identified from public reporting reviewed during this task.
Stable file hash IOC set	None identified; this is a vulnerability condition rather than a malware family or fixed exploit kit.
Best operational indicators	Python-based archive extraction followed by unexpected writes outside the intended extraction root, especially to privileged or execution-sensitive paths.

5. Threat Context

Public reporting reviewed during this task does not show confirmed in-the-wild exploitation, KEV inclusion, or named threat-actor use. That said, the vulnerability fits realistic software-supply-chain and social-engineering paths because the attacker only needs the victim to process a crafted tar archive. High-value scenarios include build systems ingesting external source archives, internal services unpacking customer submissions, developer workstations handling shared sample bundles, and automated package workflows that trust upstream tar content too readily.

The Python documentation adds an important defensive nuance: even after the fix, `filter='data'` is not presented as a complete untrusted-archive security boundary. The 3.14 `tarfile` documentation explicitly warns that the filter does not eliminate all archive-related risks and recommends additional controls such as extracting to a fresh temporary directory, disallowing symlinks when unnecessary, and enforcing OS-level resource limits. Defenders should therefore treat patching as necessary but not sufficient.

Downstream packaging also matters. Debian's tracker shows vulnerable package states for multiple Python and PyPy lines until distro-specific fixes land, which means enterprises should verify package-manager backports rather than relying only on upstream micro-version naming. For Linux fleets, the correct remediation question is often "which distro package version contains the fix?" rather than simply "is Python 3.14.6 installed?"

6.1 Detection Coverage Summary

Signal	Telemetry Source	Analytic Goal
Python process opens attacker-controlled tar archive and writes to unexpected path outside normal extraction workspace	EDR file telemetry, Linux auditd/eBPF, application logs	Identify probable exploitation or dangerous archive-handling behavior
Python- or service-driven extraction immediately followed by writes to <code>/etc</code> , <code>/usr/local/bin</code> , application config paths, or other sensitive directories	EDR file telemetry, SIEM correlation	Surface integrity-impacting out-of-directory writes
Inventory of Python runtimes and distro packages still below fixed boundaries	Asset inventory, package-management telemetry, SBOM	Find vulnerable systems before exploitation
Services that accept uploaded tar archives and call Python extraction routines	Application inventory, code review, service telemetry	Prioritize exposed workflows for patching and sandboxing
Repeated extraction failures, <code>OutsideDestinationError</code> , or filter-related exceptions after patching	Application logs, stderr capture, audit logs	Detect blocked exploit attempts or regression risk

6.2 Microsoft Defender XDR / MDE Hunt Query

```

let lookback = 30d;
let archive_activity =
    DeviceFileEvents
    | where Timestamp > ago(lookback)
    | where InitiatingProcessFileName in~ ("python", "python3", "pypy3")
    | where FileName matches regex @"(?:i).+\.(\.tar|tgz|tar\.gz|tar\.bz2|tar\.xz)$"
    | project ArchiveTime=Timestamp, DeviceId, DeviceName, InitiatingProcessId,
        InitiatingProcessSHA1, InitiatingProcessFileName,
        InitiatingProcessCommandLine,
        ArchivePath=strcat(FolderPath, "/", FileName);
let suspicious_writes =
    DeviceFileEvents
    | where Timestamp > ago(lookback)
    | where InitiatingProcessFileName in~ ("python", "python3", "pypy3")
    | where ActionType in~ ("FileCreated", "FileModified", "FileRenamed")
    | where FolderPath startswith "/etc"
        or FolderPath startswith "/usr/local/bin"
        or FolderPath startswith "/opt"
        or FolderPath startswith "/srv"
        or FolderPath startswith "/home/"
    | project WriteTime=Timestamp, DeviceId, DeviceName, InitiatingProcessId,
        InitiatingProcessSHA1, InitiatingProcessFileName,
        InitiatingProcessCommandLine,
        WrittenPath=strcat(FolderPath, "/", FileName), ActionType;
archive_activity
| join kind=inner suspicious_writes on DeviceId, InitiatingProcessSHA1
| where WriteTime between (ArchiveTime .. ArchiveTime + 5m)
| where WrittenPath !contains "/.cache/"
| where WrittenPath !contains "/venv/"
| where WrittenPath !startswith "/tmp/"
| project ArchiveTime, WriteTime, DeviceName, InitiatingProcessFileName,
    InitiatingProcessCommandLine, ArchivePath, WrittenPath, ActionType
| order by WriteTime desc

```

6.3 Splunk Hunt Query

```

index=os sourcetype=auditd (exe="/usr/bin/python3" OR exe="/usr/bin/python" OR exe="/usr/bin/pypy3")
(
    name="*.tar" OR name="*.tgz" OR name="*.tar.gz" OR name="*.tar.bz2" OR name="*.tar.xz"
    OR name="/etc/*" OR name="/usr/local/bin/*" OR name="/opt/*" OR name="/srv/*"
)
| eval watched_path=coalesce(name, file_path, path)
| transaction host pid maxspan=5m keepevicted=true
| search watched_path="*.tar" OR watched_path="*.tgz" OR watched_path="*.tar.gz"
| search watched_path="/etc/*" OR watched_path="/usr/local/bin/*" OR watched_path="/opt/*" OR
watched_path="/srv/*"
| table _time host pid exe comm watched_path user auid tty
| sort - _time

```

6.4 Hunting Notes

- Prioritize workflows that process tar archives from email submissions, customer uploads, public package repositories, threat-intel sample exchanges, or CI artifact ingestion.
- Treat Python archive extraction running as `root`, inside privileged containers, or under service accounts with broad write access as a materially higher-risk condition.
- If application logging is available, add explicit instrumentation around archive source, extraction root, rejected link members, and any blocked `OutsideDestinationError` / `LinkOutsideDestinationError` conditions.
- For containerized builders, pair package/version inventory with filesystem-write telemetry so defenders can distinguish routine extraction from out-of-scope writes.

7. Recommendations

Priority	Owner	Recommendation	Rationale
----------	-------	----------------	-----------

Critical	Platform / Infrastructure	Patch to fixed upstream releases (`3.13.14`, `3.14.6`, `3.15.0b2`) or vendor backports carrying the same fix.	Removes the known filter-bypass primitive.
High	Linux / Endpoint Engineering	Validate distro package status using vendor advisories and package changelogs rather than upstream version numbers alone.	Downstream Python packages may receive fixes on different schedules and version strings.
High	Application Owners	Review services and tools that call `tarfile.extractall()` or related archive-extraction workflows on untrusted input.	Exposed business logic may still rely on the vulnerable primitive.
High	DevOps / CI Owners	Run archive extraction in disposable directories with least privilege and no write access to sensitive paths.	Limits blast radius if a crafted archive is processed.
High	Security Engineering	Add hunts and alerts for Python-driven archive extraction followed by writes outside expected workspaces.	Detects probable exploitation attempts and unsafe behavior.
Medium	Software Supply Chain / Governance	Reduce acceptance of unsigned or unreviewed tar archives from external sources and require provenance checks where practical.	Exploitation depends on attacker-controlled archive delivery.
Medium	Developers / App Teams	Follow Python documentation guidance even after patching: avoid unnecessary symlink extraction, inspect content before extraction, and apply OS-level resource limits.	`filter='data'` is not a complete security boundary for untrusted archives.

Immediate next steps for most enterprises:

1. Identify all Python runtimes or distro packages that still fall below the fixed boundaries.
2. Enumerate workflows that extract third-party tar archives, especially in CI/CD, package ingestion, customer-upload processing, and analyst tooling.
3. Patch or sandbox those workflows first, then validate with telemetry that extracted files remain inside expected directories.
4. Preserve logs for blocked extraction attempts after patching; they may indicate exploit probing or unsafe upstream content.

8. References

- [1] Microsoft Security Response Center: <https://msrc.microsoft.com/update-guide/vulnerability/CVE-2026-7774>
- [2] CVE Project JSON 5 API: <https://cveawg.mitre.org/api/cve/CVE-2026-7774>
- [3] NVD entry: <https://nvd.nist.gov/vuln/detail/CVE-2026-7774>

- [4] NVD JSON API: <https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2026-7774>
- [5] Python security-announce thread: <https://mail.python.org/archives/list/security-announce@python.org/thread/4FU62L2M6RMMHT2QPGQNPEHHUND7CEX5/>
- [6] CPython issue `#149486`: <https://github.com/python/cpython/issues/149486>
- [7] CPython PR `#149487`: <https://github.com/python/cpython/pull/149487>
- [8] Mainline CPython fix commit: <https://github.com/python/cpython/commit/578411982c16f753f4893532510099ef665117da>
- [9] CPython 3.14 backport commit: <https://github.com/python/cpython/commit/74cca9a92fb7d653e404843a56b8bdc7b0afdbbf>
- [10] CPython 3.13 backport commit: <https://github.com/python/cpython/commit/0478bd83d82b255e0f29f613367a59d261e7eaa2>
- [11] CPython 3.12 backport commit: <https://github.com/python/cpython/commit/0d28f5e46e151718972dfabd91205444d0037b6d>
- [12] Python 3.14 changelog entry: <https://docs.python.org/3.14/whatsnew/changelog.html>
- [13] Python `tarfile` documentation: <https://docs.python.org/3/library/tarfile.html>
- [14] Debian Security Tracker: <https://security-tracker.debian.org/tracker/CVE-2026-7774>
- [15] oss-security mirror of the disclosure: <https://seclists.org/oss-sec/2026/q2/813>